

Object-Oriented Modeling of Electronic Circuits, Spring 1405

Final Take Home Exam

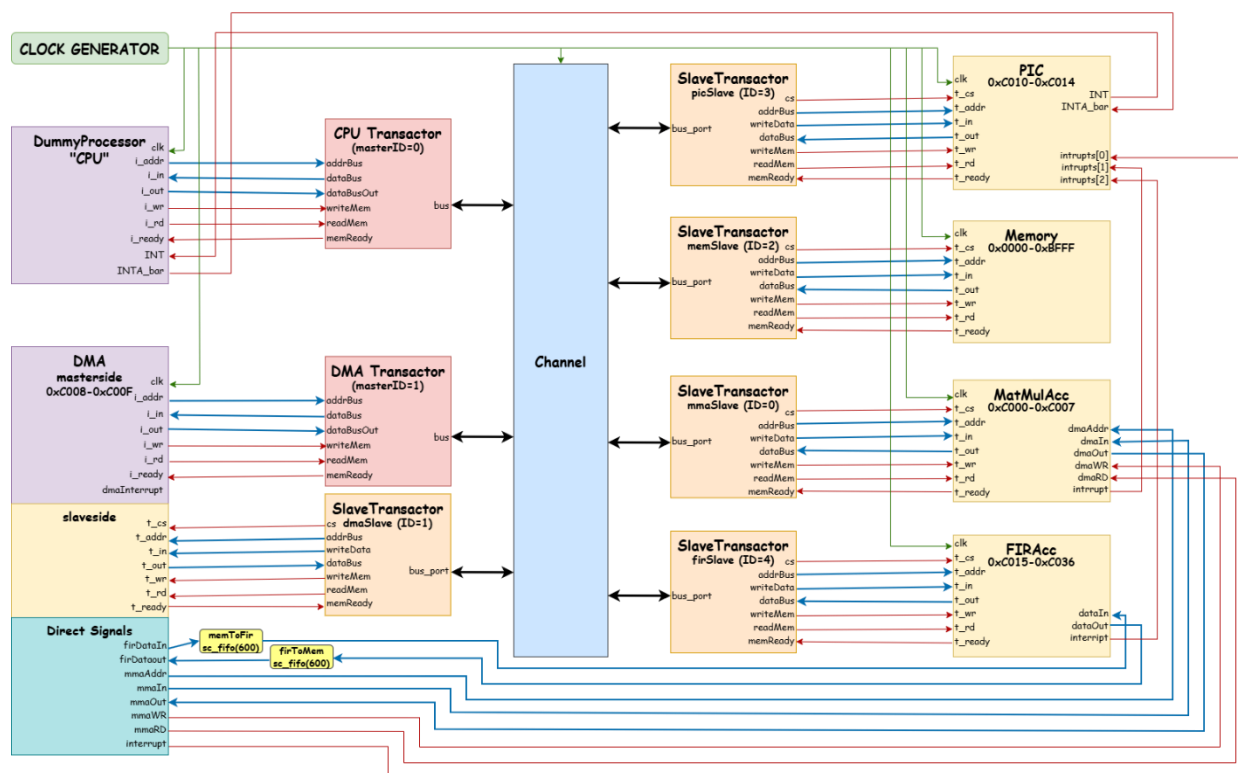
Final Report

Ali Dehghani

810103414

11.5.1405

1. Draft sketch of the system design



این دیاگرام را با استفاده از draw.io کشیدم و در واقع کل سیستم در این دیاگرام نمایش داده شده است به همراه سیگنال های ورودی و خروجی تمام بخش ها که نامگذاری آنها براساس نامگذاری در فایل system در پروژه هست.

این دیاگرام نهایی است که پیاده سازی سیستم را با استفاده از channel نشان داده است.

2. FIR accelerator BFM code(part1)

در پیاده سازی این فیلتر از پیاده سازی MMA به عنوان نمونه و قالب استفاده کردم و دقیقا از الگو و پیاده سازی شبیه به آن برای این فیلتر استفاده کردم که ساختار آن به این صورت هست:

(1) ساختار register map: برای این فیلتر 34 رجیستر اختصاص دادم که شامل این موارد است:

Local number	name	Description
0	Control	Start signal
1	Status	Is task Done
2	Chunk size	Num of samples in a process
3-33	Coefficients	Filter coefficients

- مثل MMA از دو سیگنال برای کنترل آن استفاده کردم: سیگنال کنترل که از خارج با یک کردن آن پردازش فیلتر شروع می شود دیگری یک فلگ که برای نشان دادن پایان کار فیلتر هست و این فلگ توسط خود فیلتر مقداردهی می شود و نه از خارج.
- یک رجیستر هم برای تعداد نمونه های داخل یک پارت (زیرا در پارت آخر لزوما 512 نمونه نداریم و این برای انجام فیلتر به ما کمک میکند)
- 31 رجیستر هم برای ذخیره ضرایب فیلتر

(2) ورودی و خروجی ها: دقیقا مانند MMA است تنها با این تفاوت که نمونه ها از طریق FIFO که از پیش در خود systemC وجود دارد وارد و خارج می شوند (دقیقا همانطور که صورت پروژه خواسته است)، یکی برای وارد کردن نمونه ها و یکی برای خارج کردن نمونه های فیلتر شده.

```
18.   sc_in<sc_logic> clk;
19.
20.   // Config/target port
21.   sc_in<sc_logic> t_cs;
22.   sc_in<sc_lv<N>> t_addr;
23.   sc_in<sc_lv<N>> t_in;
24.   sc_out<sc_lv<N>> t_out;
25.   sc_in<sc_logic> t_wr;
26.   sc_in<sc_logic> t_rd;
27.   sc_out<sc_logic> t_ready;
28.
29.   // Interrupt to PIC
30.   sc_out<sc_logic> interrupt;
31.
32.   // Predefined FIFO channels
33.   sc_fifo_in<sc_lv<N>> dataIn;
34.   sc_fifo_out<sc_lv<N>> dataOut;
35.
36.   // Registers
```

```

37.   sc_lv<16> controlReg;
38.   sc_lv<16> statusReg;
39.   sc_lv<16> chunkSizeReg;

```

(3) متغیرها: از دو دسته متغیر محلی هم استفاده کردم یکی برای ضرایب که پس از دریافت آنها از حافظه آنها را در یک آرایه ذخیره میکنم و یک آرایه هم برای ذخیره 30 تا از نمونه های پارت قبلی (چون برای محاسبه مقدار فیلتر شده یک نمونه به مقدار خود آن نمونه و 30 نمونه قبل آن نیاز داریم که 30 نمونه پایانی هر پارت را داخل این آرایه ذخیره میکنم تا برای پارت بعد استفاده شوند).

```

41.   int16_t coeffs[TAPS];
42.   int16_t history[TAPS - 1]; // last 30 samples carried over
    from the previous chunk

```

(4) Constructor: ابتدا مقداردهی اولیه سیگنال ها و متغیر ها (صفر کردن آنها) و بعد هم دو SC_THREAD یکی برای کانفیگ کردن فیلتر و دیگری برای انجام فیلتر کردن هر پارت.

```

44.   SC_CTOR(FIRAcc)
45.   {
46.       controlReg = 0;
47.       statusReg = 0;
48.       chunkSizeReg = MAX_CHUNK;
49.       for (int i = 0; i < TAPS; i++)
50.           coeffs[i] = 0;
51.       for (int i = 0; i < TAPS - 1; i++)
52.           history[i] = 0; // zero history before the first
    chunk
53.
54.       SC_THREAD(evalConfigReg);
55.       sensitive << clk;
56.       SC_THREAD(eval);
57.       sensitive << clk;
58.   }

```

(5) EvalConfigReg: این متد دقیقا شبیه به همین متد در MMA نوشته شده است (و فقط یکسری متغیر و آرایه متفاوت دارد) و دو حالت read, write دارد که میتواند برای کانفیگ کردن فیلتر و خواند محتوای رجیستر های آن استفاده شود.

6) Eval: این متد هم مثل MMA هست و راه اندازی و عملکرد آن (سیگنال ها) دقیقا با MMA یکسان است ولی کاری که انجام میدهد متفاوت است و به این صورت عمل میکند:

I. در ابتدا منتظر سیگنال start می ماند و بعد از دریافت آن بعضی از سیگنال ها را مقداره می کند.

```
65.          // Wait for start
66.          do
67.          {
68.              wait(clk->posedge_event());
69.          } while (controlReg[0] == '0');
70.
71.          controlReg[0] = sc_logic_0;
72.          statusReg = 0;
73.          interrupt = sc_logic_0;
```

II. بعد بررسی میکند که ورودی طول پارت برابر 512 (پارت های وسط) و یا کمتر (پارت آخر) باشد و در غیر این صورت آن را تصحیح می کند.

```
130.          int len = chunkSizeReg.to_uint();
131.          if (len > MAX_CHUNK)
132.              len = MAX_CHUNK;
133.          if (len < 0)
134.              len = 0;
```

III. بعد از آن پارت را از FIFO ورودی میخواند و داخل یک وکتور میریزد.

```
139.          std::vector<int16_t> chunk(len);
140.          for (int i = 0; i < len; i++)
141.              chunk[i] = dataIn.read().to_int();
```

IV. سپس نمونه های پارت قبل را (30 تایی که ذخیره کردیم و اگر هم همین پارت اول باشد مقدار آنها صفر است که یعنی بدون تاثیر) به آن اضافه میکند.

```
143.          // Extended window: [30 history samples |
            this chunk]
144.          std::vector<int16_t> ext(TAPS - 1 + len);
145.          for (int i = 0; i < TAPS - 1; i++)
146.              ext[i] = history[i];
147.          for (int i = 0; i < len; i++)
148.              ext[TAPS - 1 + i] = chunk[i];
```

V. حالا نمونه ها را طبق فرمول داخل فایل های پروژه فیلتر میکند با استفاده از ضرایب و بررسی میکند که مقدار اعداد به گونه ای باشد که در 16 بیت جا بشود و در اگر بزرگتر یا کوچکتر از حداقل و حداقل مقدار باشد، مقدار حداکثر یا حداقل را به جای آن قرار میدهد.

```
150.          std::vector<int16_t> yChunk(len);
151.          for (int n = 0; n < len; n++)
152.          {
153.              int64_t acc = 0;
```

```

154.             int center = TAPS - 1 + n;
155.             for (int k = 0; k < TAPS; k++)
156.                 acc += (int32_t)ext[center - k] *
(int32_t)coeffs[k];
157.
158.             acc >>= 15;
159.             if (acc > 32767)
160.                 acc = 32767;
161.             if (acc < -32768)
162.                 acc = -32768;
163.             yChunk[n] = (int16_t)acc;
164.         }

```

.VI بعد از اتمام فیلتر آنها را در FIFO خروجی می نویسد.

```

166.         // Push results to the output FIFO
167.         for (int i = 0; i < len; i++)
168.         {
169.             sc_lv<N> word;
170.             word = yChunk[i];
171.             dataOut.write(word);
172.         }

```

.VII هیستوری را برای پارت بعدی آپدیت میکند به مقادیر پایان این پارت

```

174.         // Update history
175.         for (int i = 0; i < TAPS - 1; i++)
176.             history[i] = ext[(int)ext.size() - (TAPS
- 1) + i];

```

.VIII سیگنال done را یک میکند و یک سیگنال وقفه ارسال میکند.

```

178.         statusReg = 1; // done
179.         cout << "FIR: chunk done, interrupting" <<
endl;
180.
181.         // Issue interrupt for one clock
182.         interrupt = sc_logic_1;
183.         wait(clk->posedge_event());
184.         interrupt = sc_logic_0;

```

3. DMA (part2)

من برای پیاده سازی این بخش از خود DMA موجود در کد های پردازنده که در اختیارمون قرار گرفت به عنوان پایه استفاده کردم و هر چیزی که نیاز داشتم رو به اون اضافه کردم.

DMA جدید ما هم به MMA متصل است و هم به FIR و میتواند به هر دو دیتا ارسال و دریافت کند(من قبل از پیام TA در گروه این رو طراحی کرده بود و فکر میکردم که MMA هم چون از قبل داخل کد ها بوده باید متصل باقی بمونه پس فقط FIR را به ان متصل کردم و امکان انتخاب از بین این دو برای ارسال و دریافت دیتا).

و تغییراتی که نسبت به نسخه پایه ایجاد کردم به شرح زیر هست:

1) امکان انتخاب از بین MMA و FIR: برای این کار از همان controlReg استفاده کردم و بیت چهارم آن را برای انتخاب از بین این دو تنظیم کردم یعنی زمانی که این بیت 0 باشد، MMA برای انتقال دیتا انتخاب می شود و وقتی 1 باشد FIR.

2) ارتباط با FIR: برای ارتباط با آن از همان FIFO استفاده کردم.

```
48. // Signals between DMA and FIR
49. sc_fifo_out<sc_lv<16>> firDataIn; // DMA pushes into
    FIR's input FIFO (mem -> FIR)
50. sc_fifo_in<sc_lv<16>> firDataOut; // DMA pulls from FIR's
    output FIFO (FIR -> mem)
```

3) Eval: با اضافه کردن یک شرط دیتا را به اکسلیتور مورد نظر انتقال میدهد و نحوه انتقال هر کدام متفاوت است. MMA که دقیقاً مثل قبل همانطور که در DMA پایه بوده است ولی برای FIR داخل FIFO هایی که ساختیم مینویسیم یا از انها میخوانیم.

```
165. // Check FIR is the target
166. bool toFIR = (controlReg[3] == '1');
```

4. System(part3)

برای اینکه یک سیستم قابل استفاده داشته باشیم و بتوانیم برنامه های مورد نظر را روی آن اجرا کنیم لازم بود موارد های زیر را پیاده سازی کنیم:

(1) FIR & DMA: این دو مورد را در فاز قبل پیاده سازی کردم و توضیحات مربوط به آنها را قبل تر داده بودم.

(2) MMA: از همان فایل MatMulAcc.cpp استفاده کردم بدون تغییر (هرچند در برنامه اصلی این پروژه اصلا از آن استفاده نمی شود ولی این سیستم قابلیت استفاده از آن را دارد.

(3) Bus: برای باس از نسخه پایه موجود در سیستم اولیه استفاده کردم یعنی همان Bus.h و تغییر خاصی نیاز نداشت چون تعداد initiator ها و target ها به صورت template هستند و از طریق سیستم قابل تنظیم هستند.

(4) Memory: برای مموری هم از همان فایل Memory.h استفاده کردم و فقط:

- آدرس فایل هایی که از آنها میخواند را تغییر دادم تا از فایل داخل همان پوشه بخواند.

```
52.         std::ifstream initFile("default_meminit.txt");
53.
54.         std::ifstream initFileLoad;
55.         std::ifstream PutAddr("addr.txt");
56.         std::ifstream PutData("data.txt");
57.         file = "mem.txt";
```

- زمان نوشتن در فایل dump.txt را در متد dump به گونه ای تغییر دادم تا در پایان کار پردازنده در آن بنویسد. (برای هر برنامه زمان را به گونه ای تنظیم کردم که تمام تغییرات در فایل خروجی dump وجود داشته باشد)

```
159.         wait(9999, SC_NS);
160.         out.open("dump.txt");
```

(5) PIC: در کدهایی که در اختیار ما قرار گرفته بود این ماژول فقط مختص sayac interface بود و برای dummy processor از آن استفاده نمی شد و سیگنال های وقفه به صورت مستقیم به خود dummy processor داده می شد ولی چون میخواستم که از بیرون کاملا به صورت یک پردازنده واقعی و مانند sayac به نظر برسد از آن برای dummy processor هم استفاده کردم و نیاز به تغییر خاصی نداشت و فقط باید سیگنال های مورد نظر را طبق دیاگرام برای dummy processor تعریف میکردم که از آن پشتیبانی کند.

(6) Dummy Processor: بسته به کاری که قرار است پردازنده انجام بدهد کد های مختلفی برای آن قرار دادم ولی سیگنال ها و پورت های آن کاملا ثابت و دقیقا مثل یک پردازنده واقعی است:

```
10.         sc_in<sc_logic> clk;
11.
12.         sc_out<sc_lv<16>> i_addr;
13.         sc_in<sc_lv<16>> i_in;
```

```

14.   sc_out<sc_lv<16>> i_out;
15.   sc_out<sc_logic> i_wr;
16.   sc_out<sc_logic> i_rd;
17.   sc_in<sc_logic> i_ready;
18.
19.   sc_in<sc_logic> INT;
20.   sc_out<sc_logic> INTA_bar;

```

- همچنین تنها یک SC_THREAD دارد (مثل نسخه اولیه) و آن هم doCPUStuff هست (که برای هر برنامه به صورت مجزا توضیح خواهم داد).
- یک متد readFromBus هم ساختم که برای مقداردهی سیگنال ها برای خواندن از باس هست و تقریباً شبیه writeFromBus هست که از قبل وجود داشت.

```

55.   uint16_t readFromBus(uint16_t address)
56.   {
57.       i_addr = address;
58.       i_rd = sc_logic_1;
59.       i_wr = sc_logic_0;
60.
61.       wait(clk->posedge_event());
62.       do
63.       {
64.           wait(clk->posedge_event());
65.       } while (i_ready != '1');
66.
67.       uint16_t v = i_in.read().to_uint();
68.
69.       i_rd = sc_logic_0;
70.       i_addr = 0;
71.
72.       return v;
73.   }

```

- دو متد هم برای وقفه ها که یکی برای دریافت و هندل کردن وقفه ها و دیگری برای مشخص کردن منبع وقفه ها

```

75.   int waitForInterruptAndIdentify()
76.   {
77.       // Block until the PIC raises INT, read back which
78.       // source it was (via the PIC's chosen_ISR register at
       address
79.       // PIC_BASE + PIC_NUM_PORTS), then acknowledge.
80.       wait(INT.posedge_event());
81.       uint16_t source = readFromBus(PIC_BASE +
       PIC_NUM_PORTS);
82.       INTA_bar = sc_logic_0; // acknowledge (PIC.h waits for
       this to drop)

```



```

83.     wait(clk->posedge_event());
84.     INTA_bar = sc_logic_1;
85.     return (int)source;
86. }
87.
88. // Program the PIC once so later interrupts can be told
    apart
89. void setupInterruptSources()
90. {
91.     writeToBus(PIC_BASE + 0, 1); // interrupts[0] = DMA ->
    tag 1
92.     writeToBus(PIC_BASE + 1, 2); // interrupts[1] = MMA ->
    tag 2
93.     writeToBus(PIC_BASE + 2, 3); // interrupts[2] = FIR ->
    tag 3
94. }

```

System (7) از همان فایل System.h استفاده کردم و این تغییرات را اعمال کردم:

- ابتدا باید memory map را بازنویسی میکردم زیرا در نسخه پایه مموری اصلی تنها 32 هزار عدد ظرفیت داشت و برای برنامه ای که صوت را با سرعت 0.5 خروجی میدهد کافی نیست زیرا حدودا 44 هزار عدد خواهد بود پس مموری مپ را به این صورت تغییر دادم:

اندازه	آدرس پایان	آدرس شروع	ماژول
~50000	0xBFFF	0x0000	Memory
8	0xC007	0xC000	MMA
8	0xC00F	0xC008	DMA
5	0xC014	0xC010	PIC
34	0xC036	0xC015	FIR

- در ادامه تعداد target ها را از 4 به 5 افزایش دادم و از FIR هم instant گرفتم و به باس متصل کردم.

```

214.     fir = new FIRAcc<N>("fir");
215.     fir->clk(clk);
216.     fir->interrupt(firInterrupt);
217.     fir->dataIn(memToFir);
218.     fir->dataOut(firToMem);
219.
220.     fir->t_cs(t_cs[4]);
221.     fir->t_addr(t_addr[4]);
222.     fir->t_in(t_in[4]);
223.     fir->t_out(t_out[4]);
224.     fir->t_wr(t_wr[4]);
225.     fir->t_rd(t_rd[4]);
226.     fir->t_ready(t_ready[4]);

```

```
227.         bus->t_cs[4](t_cs[4]);
228.         bus->t_addr[4](t_addr[4]);
229.         bus->t_in[4](t_in[4]);
230.         bus->t_out[4](t_out[4]);
231.         bus->t_wr[4](t_wr[4]);
232.         bus->t_rd[4](t_rd[4]);
233.         bus->t_ready[4](t_ready[4]);
234.
235.         bus->startAddress[4] = 0xC015;
236.         bus->sizeAddress[4] = 34;
```

8) TB: برای این بخش هم از همان فایل TB.cpp استفاده کردم ولی زمان شبیه سازی را به نسبت حج برنامه افزایش دادم.

5. Test system(part3)

بعد از اتمام طراحی اولیه سیستم با استفاده از bus وقت آن است که یک تست روی آن انجام دهیم و عملکرد سیستم را بررسی کنیم(که آیا FIR,DMA,Bus و بقیه موارد به درستی کار می کنند یا نه) که برای این کار از تست زیر استفاده کردم که در dummyprocessor قرار دارد و به این صورت کار میکند:

- پیکربندی اولیه کنترلر وقفه:پس از یک تأخیر اولیه ۵ کلاکی برای تثبیت سیستم، ابتدا شناسه های وقفه را در رجیسترهای کنترلر PIC ثبت می کنم تا منبع هر وقفه مشخص شود (شناسه ۱ برای DMA، شناسه ۲ برای MMA و شناسه ۳ برای FIR).
- اجرای تست ضرب ماتریس (سناریوی MMA + DMA): ورودی های دو ماتریس تستی با را در آدرس های ابتدایی حافظه RAM بارگذاری کرده و ابعاد را در رجیسترهای کنترلر شتاب دهنده MMA مقداردهی می کنم. به ماژول DMA دستور می دهم ورودی ها را از RAM به بافر داخلی MMA منتقل کند. پس از دریافت وقفه اتمام انتقال DMA از طریق PIC، فرمان شروع پردازش را به MMA ارسال می کنم. پس از دریافت وقفه پایان محاسبات MMA، نتایج ضرب را مجدداً به کمک DMA به RAM منتقل کرده و با خواندن خانه های حافظه، صحت خروجی محاسبه شده (مقدار انتظاررفته [4, 5]) را بررسی می کنم.
- اجرای تست فیلتر صوتی (سناریوی FIR + DMA): نمونه صوتی تستی را در حافظه RAM قرار داده و ضرایب فیلتر FIR را روی حالت گذر مستقیم تنظیم می کنم. با تنظیم بیت انتخاب مقصد در DMA روی حالت FIR، نمونه ها را به صف ورودی (FIFO) شتاب دهنده ارسال می کنم. پس از اتمام فیلترینگ و اعلام وقفه از سمت FIR، نتایج خروجی را از طریق صف خروجی فیلتر و به کمک DMA در بخش دیگری از حافظه RAM ذخیره می کنم.

نتایج و کد های مربوط به این تست در پوشه part3 در زیر آپلود شده برای فاز 2 موجود است و همچنین نتیجه ی این تست به جای ترمینال در فایل result.txt نوشته شده و همچنین dump.txt هم موجود است.

6. Software program(part4)

پیش از هر کاری باید خروجی wav_to_txt.cpp که به صورت اعداد صحیح هست (و البته ضرایب coefficients.txt) را به فرم باینری مکمل دو در بیاوریم که برای این کار از یک کد ++c استفاده کردم (dec_to_bin.cpp) و یک فایل txt جدید ساختم که نقش حافظه را دارد و مازول مموری در ابتدا از آن میخواند و در ابتدای آن از آدرس 0 تا 22049 را نمونه ها را قرار دادم و دقیقاً بعد از آن یعنی از آدرس 22050 ضرایب را قرار دادم. که نتایج در پوشه "Part4/dec_to_bin" قرار دارد.

با توجه به محدودیت حافظه سه برنامه مجزا برای صدای فیلتر شده، سرعت 2 برابر و سرعت نصف نوشته شده است که هر کدام را به صورت مجزا توضیح میدهم. (لازم به ذکر است که هر کدام از این سه برنامه تنها در 3 فایل با هم تفاوت دارند: TB.cpp, System.h, DummyProcessor.h که تفاوت اصلی در فایل Dummy processor است که همان پردازنده است و در فایل های دیگر تنها فایلی که include شده است نام متفاوتی دارد.):

1) Filter:

- در این مرحله پردازنده نقش کار فیلترینگ سیگنال صوتی را انجام میدهد.
- کل اجرای برنامه در تابع doCPUStuff انجام می شود و شامل سه گام اصلی زیر است :
- I. بارگذاری ضرایب فیلتر: پس از آماده سازی و پیکربندی کنترلر وقفه و گذشت 5 سیکل کلاک، ابتدا ۳۱ ضریب فیلتر را از حافظه از طریق باس می خوانم. سپس این 31 ضریب را به ترتیب در رجیسترهای داخلی شتاب دهنده می نویسم تا فیلتر برای پردازش آماده شود .
- II. پردازش نمونه ها: در این مرحله نمونه های موجود در حافظه را به کمک DMA به فیلتر میفرستم و دستور پردازش را هم به فیلتر ارسال میکنم و برای هر بلاک، مراحل Handshaking زیر را انجام می دهیم :
- ابتدا طول بلاک را در رجیستر تنظیمات فیلتر نوشته و بیت شروع کار آن را فعال می کنم .
 - به کنترلر DMA دستور می دهیم که نمونه های نويزدار را از حافظه به صف ورودی (FIFO) شتاب دهنده فیلتر منتقل کند و منتظر وقفه اتمام کار DMA می مانم .
 - پس از پایان انتقال، منتظر دریافت وقفه از سمت شتاب دهنده فیلتر می مانم تا مطمئن شوم فیلترینگ بلاک جاری تمام شده است .
 - در نهایت، مجدداً DMA را تنظیم می کنم تا نمونه های فیلتر شده را از صف خروجی فیلتر بخواند و دقیقاً در همان آدرس های قبلی در حافظه (In-Place Overwrite) ذخیره کند .
- III. استخراج و ذخیره نتایج: پس از اتمام پردازش هر 44 بلاک، تمام 22050 نمونه صوتی فیلتر شده را یکی یکی از آدرس صفر حافظه RAM می خوانم. داده های خوانده شده را در یک فایل خروجی به نام filtered.txt می نویسم تا در مرحله بعد توسط برنامه txt_to_wav.cpp به فایل صوتی WAV تبدیل شود.

2) Filtered_x2:

در این مرحله ماژول پردازنده وظیفه کاهش نرخ نمونه برداری به نصف را برعهده دارد. در این برنامه برخلاف مراحل قبل، تمام عملیات صرفاً توسط خود پردازنده انجام می‌شود و از هیچ‌کدام از شتاب‌دهنده‌های سخت‌افزاری (MMA، FIR، DMA و کنترلر وقفه PIC) استفاده نمی‌کنم. پورت‌های مربوط به این ماژول‌ها در کد حفظ شده‌اند تا سازگاری با فایل‌های مشترک سیستم (System.h، Bus.h و Memory.h) حفظ شود، اما هیچ عملیاتی روی آن‌ها انجام نمی‌گیرد. کل روند اجرای برنامه در تابع doCPUStuff انجام می‌شود و شامل دو گام اصلی زیر است:

I. پس از گذشت 5 سیکل کلاک برای تثبیت سیستم، من 22050 نمونه صوتی فیلترشده را که از آدرس صفر حافظه شروع می‌شوند پردازش می‌کنم. الگوریتم کار به این صورت است که یک‌درمیان نمونه‌ها را حذف می‌کنم. یعنی فقط نمونه‌های موجود در اندیس‌های زوج را نگه می‌دارم که باعث می‌شود تعداد نمونه‌ها به نصف، یعنی 11025 نمونه کاهش یابد. این عملیات را به صورت بازنویسی در همان مکان (In-Place) و بدون نیاز به حافظه اضافی انجام می‌دهم. از آنجا که پردازش به صورت رو به جلو (از اندیس کم به زیاد) انجام می‌شود و همواره شرط $i \leq 2i$ برقرار است، نوشتن داده جدید در آدرس i همیشه پس از خوانده شدن داده قبلی از آدرس $2i$ اتفاق می‌افتد. در نتیجه هیچ تداخلی در داده‌های حافظه ایجاد نمی‌شود.

II. پس از اتمام عملیات کاهش نمونه‌ها، تمام نمونه‌های خروجی را یکی‌یکی از آدرس‌های ابتدایی حافظه می‌خوانم. داده‌های خوانده‌شده را در یک فایل متنی جدید به نام filtered_2x.txt ذخیره می‌کنم. در نهایت، با فراخوانی تابع sc_stop شبیه‌سازی سیستم را به طور کامل خاتمه می‌دهم.

3) Filtered_Half:

در این مرحله ماژول پردازنده وظیفه افزایش تعداد نمونه‌ها به دو برابر را برعهده دارد. در این مرحله نیز همانند مرحله قبل، تمام پردازش‌ها صرفاً توسط خود پردازنده انجام می‌شود و پورت‌های مربوط به شتاب‌دهنده‌های سخت‌افزاری (MMA، FIR، DMA و PIC) فقط برای حفظ سازگاری با ماژول‌های مشترک سیستم (System.h، Bus.h و Memory.h) متصل شده‌اند و هیچ استفاده‌ای از آن‌ها نمی‌کنم. کل روند اجرای برنامه در تابع doCPUStuff انجام می‌شود و شامل دو گام اصلی زیر است:

I. پس از گذشت 5 سیکل کلاک برای تثبیت اولیه سیستم، من 22050 نمونه صوتی فیلترشده را که در حافظه از آدرس صفر قرار دارند پردازش می‌کنم. الگوریتم کار به این صورت است که میانگین هر دو نمونه متوالی را محاسبه کرده و بین همان دو نمونه درج می‌کنم. با این کار تعداد کل نمونه‌های خروجی دو برابر شده و به 44100 نمونه می‌رسد. برای اینکه بتوانم این عملیات را در همان مکان اولیه حافظه و بدون نیاز به فضای رم اضافی انجام دهم، پردازش را از آخر به اول انجام می‌دهم:

■ در هر مرحله i ، مقدار نمونه فعلی و نمونه بعدی را از باس می‌خوانم. برای آخرین نمونه که نمونه بعدی ندارد، مقدار نمونه بعدی را برابر با خود همان نمونه فرض می‌کنم تا میانگین با خودش برابر شود. سپس مقدار اصلی را در آدرس $2i$ و میانگین محاسبه‌شده را در آدرس $2i+1$ می‌نویسم. از آنجا که همواره رابطه $i \leq 2i$ برقرار است و حلقه از اندیس‌های بزرگ به سمت اندیس‌های کوچک حرکت می‌کند، آدرس‌های هدف نوشتن همیشه بزرگتر یا مساوی آدرس‌هایی هستند که در مراحل بعدی حلقه باید خوانده شوند؛ در نتیجه هیچ نمونه‌خوانده‌ای رونویسی یا تخریب نمی‌شود.

II. پس از اتمام دو برابر شدن نمونه‌ها، تمام نمونه‌های خروجی را از آدرس صفر حافظه تا انتها می‌خوانم. نمونه‌های خوانده‌شده را در یک فایل متنی جدید به نام `filtered_half.txt` ذخیره می‌کنم. در نهایت، با فراخوانی دستور `sc_stop` پایان کار را اعلام کرده و شبیه‌سازی سیستم را خاتمه می‌دهم.

پس از آماده شدن فایل `txt` هر سه مرحله حالا باید نتیجه را بررسی کنیم که برای بررسی نتیجه از یک کد `c++` استفاده کردم (`reference_filter.cpp`) که به صورت کاملاً نرم افزاری نمونه‌های نویزی را با استفاده از ضرایب و دقیقاً همان الگوریتم اصلی فیلتر، فیلتر کند و علاوه بر آن نسخه‌ی `x2` و `half` را هم بسازد و در سه فایل `txt` مجزا بریزد. سپس با استفاده از یک دستور `diff` در ترمینال لینوکس می‌توانیم خروجی پردازنده را با خروجی کد بالا مقایسه کنیم که درستی نتیجه را تایید کند. که نتایج در پوشه `"Part4/validate"` قرار دارد.

و در آخر برای تبدیل نمونه‌ها فیلتر شده به صدا هم از کدی `txt_to_wav.cpp` که در اختیارمون قرار گرفته بود استفاده کردم که نتیجه در پوشه `"Part4/converter"` قرار دارد.

7. Channel

برای پیاده سازی این بخش از کد های سیستم پایه (System with channel interface) استفاده کردم که در اختیارمون قرار گرفته بود و برای این کار فایل های مربوط به چنل را به صورت زیر با فایل Bus.h جایگزین کردم:

- interfaceClasses.h: بدون تغییر
- SlaveTransactor.cpp: بدون تغییر
- SlaveTransactor.h: بدون تغییر
- Transactor.cpp: بدون تغییر
- Transactor.h: بدون تغییر
- busChannel.h

- (1) افزایش تعداد target ها از 4 به 5 (به خاطر اضافه شدن فیلتر)
 - (2) اضافه کردن مموری مپ جدید به صورت یک آرایه (که در cpp آن استفاده می شود)
- busChannel.cpp

- (1) باز نویسی decodeSlave به خاطر اضافه شدن فیلتر.
- (2) در متد slaveMMcollection() آدرس به صورت آدرس گلوبال به مازول های هدف ارسال میشد در حالی که در نسخه باس آنها را به صورت محلی با کم کردن آدرس پایه ارسال میکردم به همین خاطر آدرس ارسالی را درست کردم.

و بر روی فایل های سیستم part4 (software program) هم این تغییرات را اعمال کردم تا با چنل هماهنگ باشند (فایل هایی که نامی از آنها برده نشده تغییری نکرده اند و علامت * به خاطر مشترک بودن علت باگ هست که در صفحه بعد علت آن باگ توضیح داده شده است):

- DMA.cpp
- (1) اضافه کردن wait() در ابتدای شاخه خواندن evalConf() *
- (2) در حلقه انتقال نمونه ها به داخل و خارج فیلتر بعد از انتقال هر کلمه چه نوشتن و چه خواندن یک wait() اضافه کردم *
- FIR.h: اضافه کردن wait() در ابتدای شاخه خواندن evalConfigReg() *
- PIC.h
- (1) اضافه کردن wait() در ابتدای شاخه خواندن evalConfigReg() *
- (2) در متد eval() در حلقه انتخاب وقفه گاهی اوقات فیلتر سریع تر از DMA کارش تمام میشد و سیگنال وقفه را ارسال میکرد (چون تعداد wait() های فیلتر کمتر از DMA است) PIC برای وقفه فیلتر را انتخاب میکرد و به همین خاطر چون سیگنال وقفه DMA بعد از آن میرسید ولی سیستم وارد یک انتظار طولانی میشد این سیگنال که یک سیکل 1 شده از دست میرفت و برای از بین بردن این مشکل یک متد جدید اضافه کردم که وظیفه

لچ کردن این سیگنال های وقفه را دارد(latchInterrupts) تا هیچکدام از آنها از بین نرود.

- DummyProcessor.h: اضافه کردن wait() به انتهای متد های readFromBus() و * writeToBus()
- System.h: به جای نمونه گیری از باس از ساختار چنل استفاده کردم و اتصالات را با آن (busChannel)+5(SlaveTransactor)+2(Transactor) ایجاد کردم که نتیجه آن دقیقاً مثل دیاگرام صفحه اول گزارش است.

*با تغییر راه ارتباطی بخش های مختلف سیستم از باس به چنل(طبق پیاده سازی که از چنل داشتم) یک باگ مهم تقریباً در کل سیستم ایجاد شد که این مشکل در سیستم با باس وجود نداشت. این مشکل به این صورت بود که در پیاده سازی باس مقدار سیگنال های مختلف(مثل ready) دقیقاً هر کلاک بررسی می شد در حالی که در پیاده سازی چنل بررسی سیگنال ها به صورت event based بود و تنها زمانی که مقدار یک سیگنال تغییر میکرد(به طور قابل مشاهده) متد را بیدار میکرد یا کاری که باید را انجام میداد، برای مثال:

در arbiter نسخه باس داشتیم:

```
247. // Wait for a request
248. do
249. {
250. wait(clk->posedge_event());
251. } while (reqs0red != '1');
```

ولی در SlaveTransactor داریم:

```
29.SC_METHOD(on_mem_ready); // Watch for memReady signal
30.sensitive << memReady;
```

```
32.void SlaveTransactor::on_mem_ready() {
33.    if (memReady.read() == SC_LOGIC_1) {
34.
35.        sc_lv<16> read_value = dataBus.read();
36.
37.        bus_port->slaveMMresponse(read_value);
38.
39.        slave_done.notify();
40.    }
41.}
```

در نسخه باس در هر کلاک بررسی میکند که آیا مقدار مورد نظر را دارد یا نه ولی در چنل متد بالا را نسبت به تغییر مقدار memReady حساس کردم که به صورت عمل میکند که SystemC در پایان هر دلتاسایکل، مقدار نهایی سیگنال (بعد از اعمال آخرین نوشتن ثبت شده) را با مقدار قبلی از آن

دلتاسایکل مقایسه می‌کند. فقط اگر این دو متفاوت باشند، event رخ میدهد و متدهای منتظر بیدار می‌شوند. در باگ ما، چون سیگنال از قبل مقدارش 1 بود، وسط راه به 0 نوشته شد ولی بلافاصله بدون wait() دوباره 1 شد مقدار نهایی (1) دقیقاً با مقدار قبل از دلتاسایکل (1) یکی است، پس از دید systemC هیچ تغییری رخ نداده.

این دقیقاً معادل تفاوت بین یک لچ سطح‌محور (level-sensitive latch) و یک فلیپ فلاپ لبه‌محور (edge-triggered FF) در برابر یک گلیچ کوتاه است: لچ چون همیشه مقدار فعلی ورودی را منعکس می‌کند، یک گلیچ خیلی کوتاه را هم می‌بیند؛ ولی فلیپ‌فلاپ فقط دقیقاً سر لبه‌ی ساعت نمونه‌برداری می‌کند و اگر گلیچ بین دو لبه شروع و تمام شود، اصلاً نمی‌بیندش. باس رفتار لچ داشت چنل ما رفتار فلیپ فلاپ داشت و همین تفاوت، منشأ تمام این باگ‌ها بود.

برای رفع این مشکل باید کاری میکردم که سیگنال‌ها به اندازه کافی فعال بمانند و به همین خاطر با اضافه کردن wait() باعث شدم به اندازه کافی فعال بمانند (منظورم از به اندازه کافی چیه؟ systemC زمانی که نوشتن یا تغییری روی یک سیگنال انجام می‌شود همان لحظه آن تغییر را روی آن سیگنال اعمال نمی‌کند و در پایان delta-cycle آن را در سیگنال ذخیره می‌کند و اگر هیچ wait() بین دو نوشتن وجود نداشته باشد که باعث شود سیگنال دوبار نوشته شود فقط آخرین تغییر را اعمال میکند (چون همه‌ی این اتفاقات در یک دلتا سیکل افتاده است)).

8. Requested

برای نسخه نهایی پروژه موارد زیر خواسته شده که هر کدام را در آدرس های زیر قرار دادم:

- Clean sound => Clean/filtered.wav
- Clean 2× sound => Clean /filtered_2x.wav
- Clean 0.5× sound => Clean /filtered_half.wav
- Proposed system code => Systems
- Illustration of the system => Diagrams
- A report showing the flow of the work => Report.pdf

همچنین ساختار فایل های نهایی به این صورت است:

- Clean: نسخه فیلتر شده در هر سه سرعت به همراه صدای نویز دار برای مقایسه قرار دارد.
- Converter: دو برنامه جدا یکی برای تبدیل متن به صدا (برای تبدیل نتیجه پردازش به صدا) و دیگری برای تبدیل اعداد صحیح به باینری مکمل دو (برای ورودی مموری سیستم)
- Diagrams: دو دیاگرام یکی با پیاده سازی باس و دیگری با پیاده سازی چنل که هر دو به وسیله ی drawio رسم شده اند.
- Systems: دو سیستم جدا یکی با پیاده سازی باس و دیگری با پیاده سازی چنل که هر کدام شامل سه پوشه هستند که در واقع تنها تفاوتشان برنامه ایست که اجرا میکنند (filter , x2 , x0.5)
- Validate: یک کد C++ که به صورت نرم افزاری کار فیلتر و x2 و x0.5 را انجام می دهد برای مقایسه صحت نتیجه سیستم (که میتوان از دستور diff در ترمینال لینوکس استفاده کرد و فایل خروجی سیستم با رفرنس برنامه این پوشه رو مقایسه کرد)
- Report.pdf: فایل گزارش کامل